ANÁLISIS DEL RETO

1.Nicolás Miguel Murillo Cristancho, n.murilloc@uniandes.edu.co, 202213891

2. Juan Pablo Peña Jaime, j.penaj@uniandes.edu.co, 202212756

3. Sara Sofía Cárdenas Rodríguez, ss.cardenas@uniandes.edu.co, 202214907

Carga de datos

Descripción

Entrada	Archivo de datos
Salidas	Los 3 primeros y últimos videos encontrados
Implementado (Sí/No)	Si se implementó, hecho por el grupo en conjunto.

Análisis de complejidad

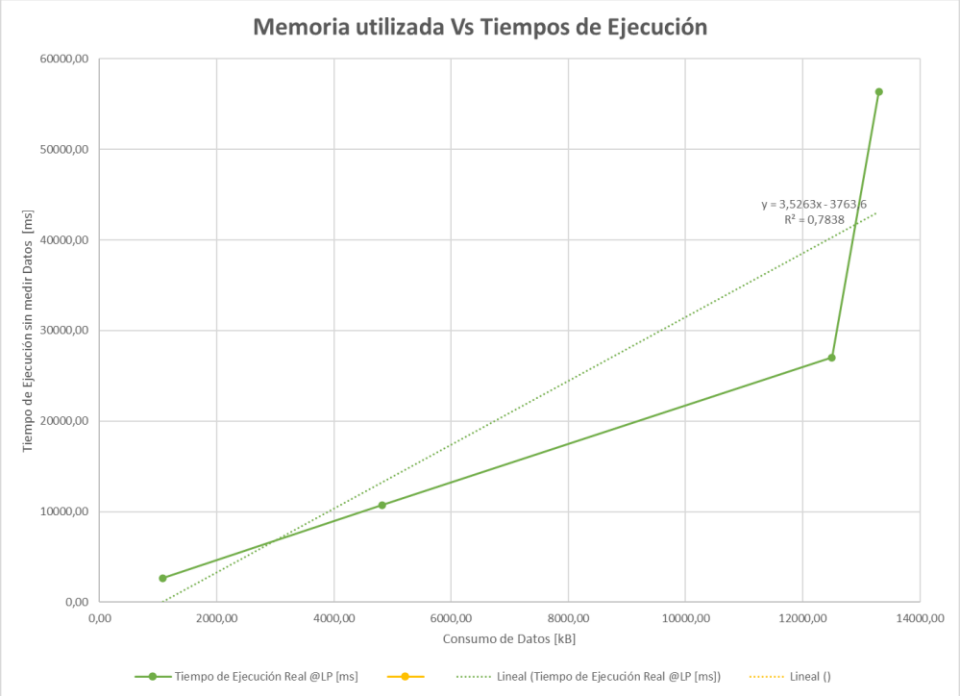
RETO 1		**RETO 2**	
Pasos	**Complejidad**	**Pasos**	**Complejidad**
1. Crear catalog con 3 listas	O (1)	1. Crear catalog con 6 índices	O(1)
2. Añadir videos	O(n)	2. Añadir videos a la lista de videos y en cada map (m son los valores separados por coma)	O(n*m)
3. Merge Sort	O(n log n)	3. Merge Sort a la lista de videos	O(n log n)
TOTAL	***O(n log n)***	***TOTAL***	***O(n*m log n)***

Pruebas Realizadas

RETO 1 (Carga)		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large - Array list	5368.206	51788.944
50% - Array list	2437.375	25874.657
Large - Single linked	420010.965	819880.143
50% - Single linked	24500.621	520220.821

RETO 2 (Carga)		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large	13301.193	56344.190
50%	12500.520	27010.060
20%	4824.870	10739.462
5%	1081.650	2664.223

Gráficas



Análisis

Complejidad: La complejidad aumentó ligeramente porque al cargar cada video se invocaban funciones para añadir su información al map respectivo. Sin embargo, esto hará que la complejidad de los requerimientos tienda a ser O(1)

Tiempo de ejecución: El tiempo depende del tamaño del archivo y aumentó ligeramente para conformar los mapas. A futuro, esto hará que los tiempos en los requerimientos no dependan de una iteración y sean prácticamente independientes a la cantidad de datos.

Memoria: Es ligeramente superior a cuando se usó Array List en el reto 1. Pero en los requerimientos, hará que el gasto de memoria sea despreciable.

Requerimiento 1

Descripción

Entrada	Maps, Año
Salidas	Las 3 primeras y últimas películas encontradas
Implementado (Sí/No)	Si se implementó, hecho por el grupo en conjunto.

Análisis de complejidad

RETO 1		**RETO 2**	
Pasos	**Complejidad**	**Pasos**	**Complejidad**
1. Asignación de variables	O (1)	1. Asignación de variables	O(1)

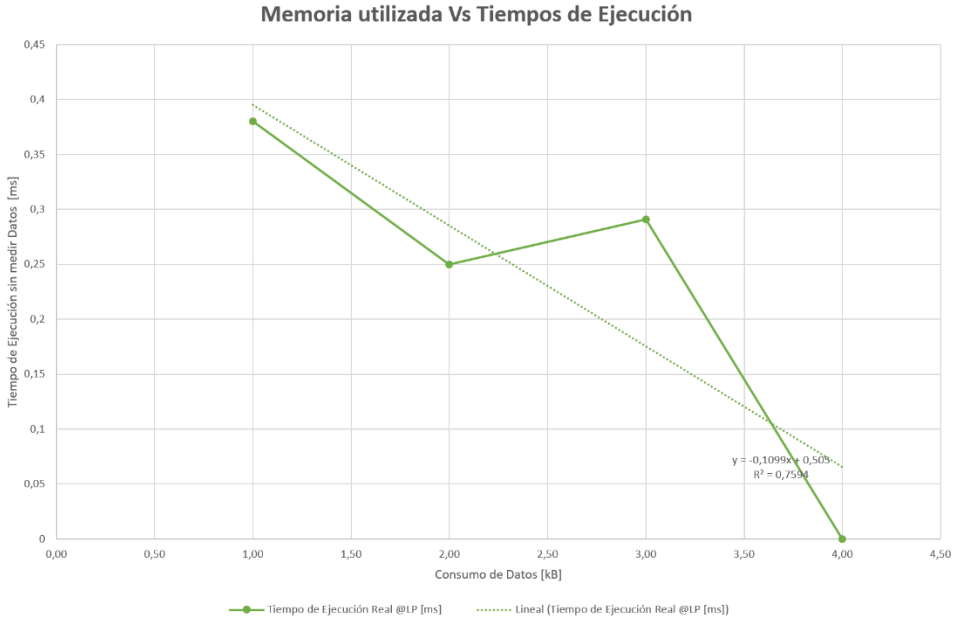
2. Iteración	O(n)	2. Obtener value (lista)	O(1)
3. Merge sort	O(n log n)	3. Merge sort	O(n log n)
TOTAL	***O(n log n)***	***TOTAL***	***O(n log n)***

Pruebas Realizadas

RETO 1 (1920 – 1999)		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large - Array list	0,460	221
50% - Array list	0,300	232
Large - Single linked	1,115	230
50% - Single linked	0,470	208

RETO 2 (1999) Chaining		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large	0,380	0,461
50%	0,250	0,461
20%	0,291	0,461
5%	0,365	0,461

Gráficas



Análisis

Complejidad: Sin tener en cuenta el merge sort, la complejidad pasó de O(n) a O(1)

Tiempo de ejecución: En el reto 2, se redujo el tiempo en un 25%, al no existir iteración

Requerimiento 2

Descripción

Entrada	Maps, Fecha que se quiere consultar
Salidas	Los 3 primeros y últimos shows encontrados
Implementado (Sí/No)	Si se implementó, hecho por el grupo en conjunto.

Análisis de complejidad

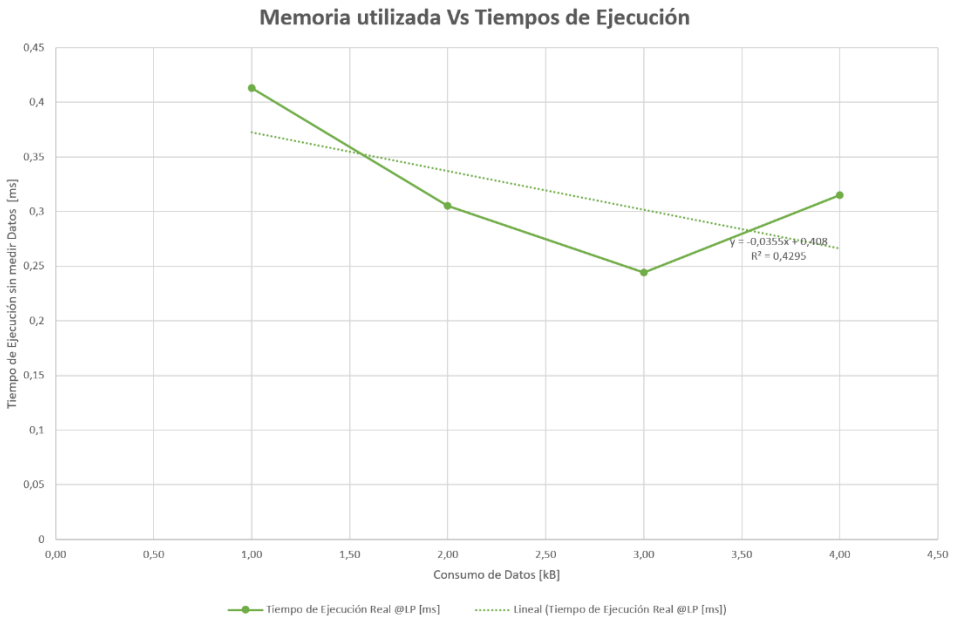
RETO 1		**RETO 2**	
Pasos	**Complejidad**	**Pasos**	**Complejidad**
1. Asignación de variables	O (1)	1. Asignación de variables	O(1)
2. Iteración	O(n)	2. Obtener value (lista)	O(1)
3. Merge sort	O(n log n)	3. Merge sort	O(n log n)
TOTAL	***O(n log n)***	***TOTAL***	***O(n log n)***

Pruebas Realizadas

RETO 1 (January 07, 2018 – December 31, 2020)		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large - Array list	0,983	258
50% - Array list	0,525	202
Large - Single linked	0,997	193
50% - Single linked	0,394	170

RETO 2 (November 12, 2019) Chaining		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large	0,413	0,461
50%	0,305	0,461
20%	0,244	0,461
5%	0,315	0,461

Gráficas



Análisis

Complejidad: Sin tener en cuenta el merge sort, la complejidad pasó de O(n) a O(1)

Tiempo de ejecución: En el reto 2, se redujo el tiempo en un 50%, al no existir iteración

Requerimiento 3

Descripción

Entrada	Maps, Actor a consultar
Salidas	Los 3 primeros y últimos videos encontrados
Implementado (Sí/No)	Si se implementó, Nicolás Murillo.

Análisis de complejidad

RETO 1		**RETO 2**	
Pasos	**Complejidad**	**Pasos**	**Complejidad**
1. Asignación de variables	O (1)	1. Asignación de variables	O(1)
2. Iteración	O(p*s)	2. Obtener value (lista)	O(1)
3. Merge sort	O(n log n)	3. Merge sort	O(n log n)
TOTAL	***O(*n log n*)***	***TOTAL***	***O(n log n)***

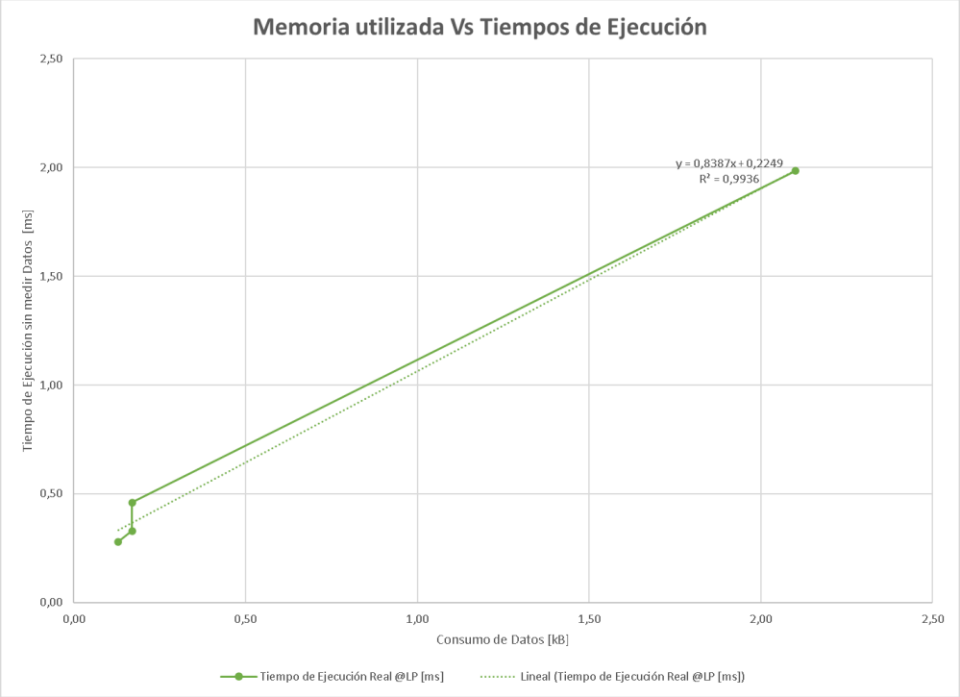
Pruebas Realizadas

RETO 1 (Bing Crosby)		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large - Array list	0.310	209
50% - Array list	0.110	203
Large - Single linked	8.862	209

50% - Single linked	2.472	190

RETO 2 (Bing Crosby) Probing		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large	1.985	2.102
50%	0.461	0.172
20%	0.331	0.170
5%	0.289	0.130

Gráficas



Análisis

Complejidad: respecto al reto 1, se redujo la complejidad temporal pues ya no es necesario iterar si no que ahora basta con obtener ciertos datos en un tiempo constante O(1) gracias a los diccionarios y tablas de hash. Aunque se debe seguir usando el merge sort que hace que la complejidad total sea n log (n)

Tiempo de ejecución: En el reto 2, se redujo el tiempo en un 40%, al no existir iteración

Si bien se observa que los tiempos de ejecución disminuyen en promedio respecto al reto 1, estamos hablando de diferencias de unidades de milisegundos, y como hay cierto margen de error en la medición, la diferencia es casi despreciable.

Requerimiento 4

Descripción

Entrada	Maps, Género
Salidas	Los 3 primeros y últimos videos encontrados
Implementado (Sí/No)	Si se implementó, Juan Pablo Peña.

Análisis de complejidad

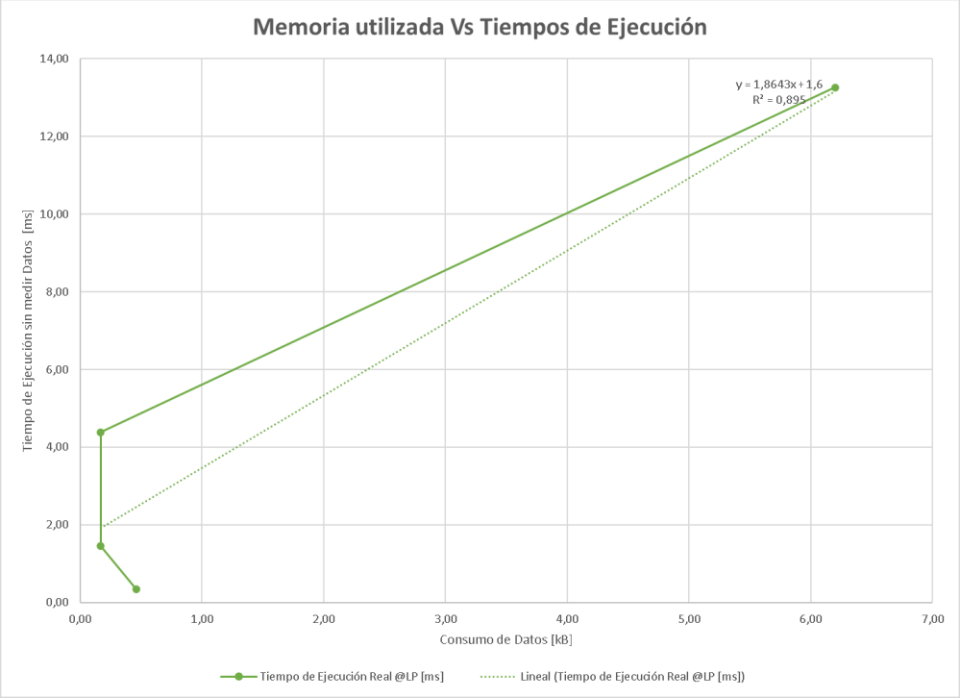
RETO 1		**RETO 2**	
Pasos	**Complejidad**	**Pasos**	**Complejidad**
1. Asignación de variables	O (1)	1. Asignación de variables	O(1)
2. Iteración	O(p*s)	2. Obtener value (lista)	O(1)
3. Merge sort	O(n log n)	3. Merge sort	O(n log n)
TOTAL	***O(*n log n*)***	***TOTAL***	***O(n log n)***

Pruebas Realizadas

RETO 1 (Fantasy)		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large - Array list	0.350	206
50% - Array list	0.170	231
Large - Single linked	8.876	220
50% - Single linked	2.627	182

RETO 2 (Fantasy) Probing		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large	13.265	6.277
50%	4.396	0.172
20%	1.454	0.172
5%	0.353	0.461

Gráficas



Análisis

Complejidad: respecto al reto 1, se redujo la complejidad temporal pues ya no es necesario iterar si no que ahora basta con obtener ciertos datos en un tiempo constante O(1) gracias a los diccionarios y tablas de hash. Aunque se debe seguir usando el merge sort que hace que la complejidad total sea n log (n)

Tiempo de ejecución: En el reto 2, se redujo el tiempo en un 70%, al no existir iteración

Si bien se observa que los tiempos de ejecución disminuyen en promedio respecto al reto 1, estamos hablando de diferencias de unidades de milisegundos, y como hay cierto margen de error en la medición, la diferencia es casi despreciable.

Requerimiento 5

Descripción

Entrada	Maps, País
Salidas	Los 3 primeros y últimos videos encontrados
Implementado (Sí/No)	Si se implementó, Sara Cárdenas

Análisis de complejidad

RETO 1		**RETO 2**	
Pasos	**Complejidad**	**Pasos**	**Complejidad**
1. Asignación de variables	O(1)	1. Asignación de variables	O(1)

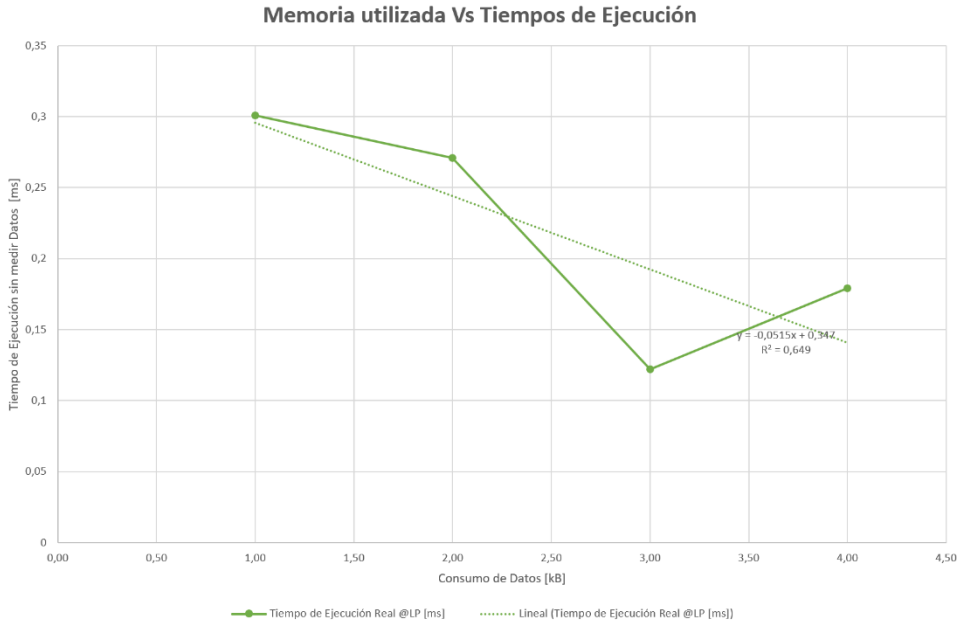
2. Iteración	O(p*s)	2. Obtener value (lista)	O(1)
3. Merge sort	*O(*n log n*)*	3. Merge sort	O(n log n)
TOTAL	**O(p*s log n)**	***TOTAL***	***O(n log n)***

Pruebas Realizadas

RETO 1 (Germany)		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large - Array list	1.270	232
50% - Array list	0.980	234
Large - Single linked	8.831	215
50% - Single linked	2.489	187

RETO 2 (Germany) Probing		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large	0,301	0,461
50%	0,271	0,461
20%	0,122	0,461
5%	0,179	0,461

Gráficas



Análisis

Complejidad: respecto al reto 1, se redujo la complejidad temporal pues ya no es necesario iterar si no que ahora basta con obtener ciertos datos en un tiempo constante O(1) gracias a los diccionarios y

tablas de hash. Aunque se debe seguir usando el merge sort que hace que la complejidad total sea n log (n)

Tiempo de ejecución: En el reto 2, se redujo el tiempo en un 75%, al no existir iteración

Si bien se observa que los tiempos de ejecución disminuyen en promedio respecto al reto 1, estamos hablando de diferencias de unidades de milisegundos, y como hay cierto margen de error en la medición, la diferencia es casi despreciable.

Requerimiento 6

Descripción

Entrada	Maps, Director
Salidas	Los 3 primeros y últimos videos encontrados
Implementado (Sí/No)	Si se implementó, hecho por el grupo en conjunto.

Análisis de complejidad

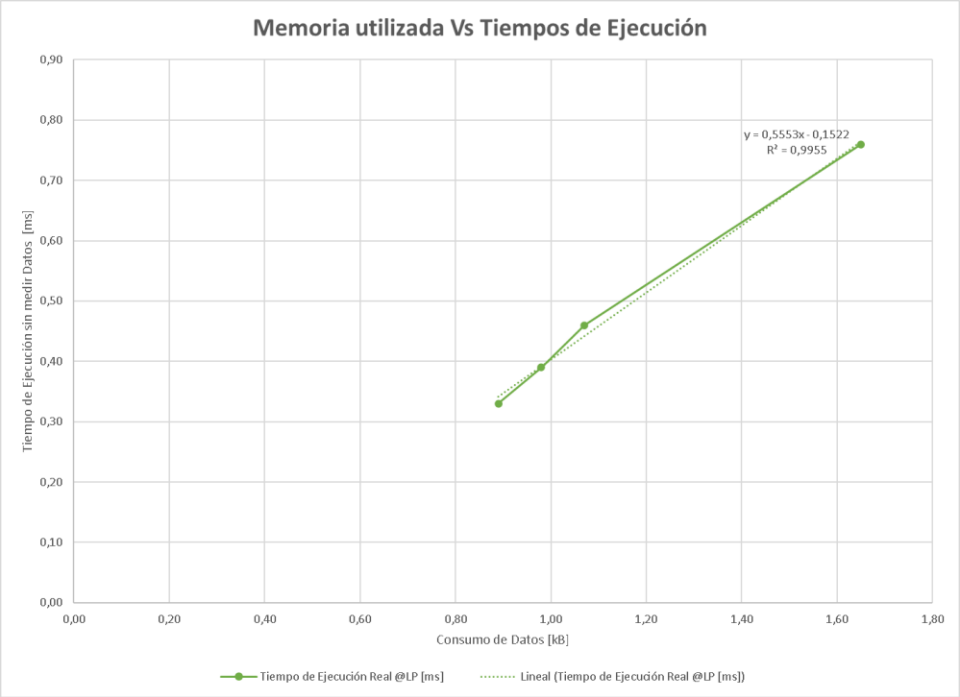
RETO 1		**RETO 2**	
Pasos	**Complejidad**	**Pasos**	**Complejidad**
1. Asignación de variables	O (1)	1. Asignar y obtener values	O(1)
2. Iteración Películas + Shows	O(n*g)	2. Obtener géneros	O(n*g)
3. Merge sort	O(n log n)	3. Merge sort	O(n log n)
TOTAL	***O(n*g*log n)***	***TOTAL***	***O(n*g*log n)***

Pruebas Realizadas

RETO 1 (John Hughes)		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large - Array list	0.180	207
50% - Array list	0.150	212
Large - Single linked	8.840	211
50% - Single linked	2.519	185

RETO 2 (John Hughes) Probing		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large	0.763	1.652
50%	0.462	1.074
20%	0.394	0.988
5%	0.332	0.890

Gráficas



Análisis

Complejidad: respecto al reto 1, se redujo la complejidad temporal pues ya no es necesario iterar si no que ahora basta con obtener ciertos datos en un tiempo constante O(1) gracias a los diccionarios y tablas de hash. Aunque se debe seguir usando el merge sort que hace que la complejidad total sea n log (n)

Tiempo de ejecución: En el reto 2, se redujo el tiempo en un 40%, al no existir iteración

Si bien se observa que los tiempos de ejecución disminuyen en promedio respecto al reto 1, estamos hablando de diferencias de unidades de milisegundos, y como hay cierto margen de error en la medición, la diferencia es casi despreciable.

Requerimiento 7

Descripción

Entrada	Maps, Número del top
Salidas	El ranking por género
Implementado (Sí/No)	Si se implementó, hecho por el grupo en conjunto.

Análisis de complejidad

RETO 1		**RETO 2**	
Pasos	**Complejidad**	**Pasos**	**Complejidad**
1. Asignación de variables	O (1)	1. Asignación de variables	O(1)
2. Iteración Videos	O(n*g)	2. Obtener values (lista y conteo)	O(1)
3. Merge sort	O(n log n)	3. Merge sort	O(n log n)

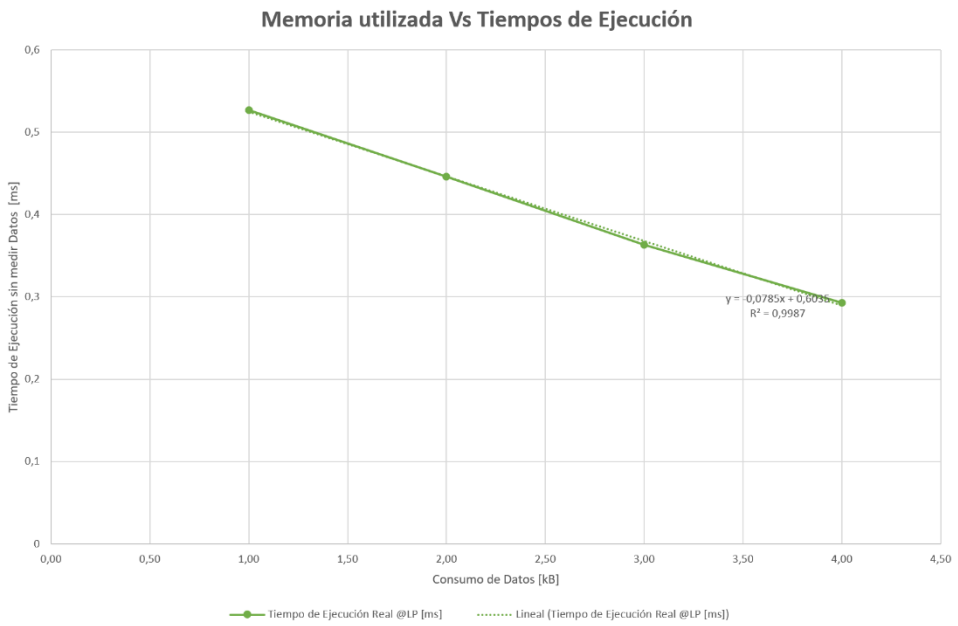
TOTAL	***O(n*g*log n)***	***TOTAL***	***O(n log n)***

Pruebas Realizadas

RETO 1 (5)		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large - Array list	0.470	258
50% - Array list	0.227	214
Large - Single linked	19.523	198
50% - Single linked	4.885	191

RETO 2 (5) Probing		
Entrada	**Tiempo (ms)**	**Memoria (kB)**
Large	0,527	0,461
50%	0,446	0,461
20%	0,363	0,461
5%	0,293	0,461

Gráficas



Análisis

Complejidad: respecto al reto 1, se redujo la complejidad temporal pues ya no es necesario iterar si no que ahora basta con obtener ciertos datos en un tiempo constante O(1) gracias a los diccionarios y tablas de hash. Aunque se debe seguir usando el merge sort que hace que la complejidad total sea n log (n)

Tiempo de ejecución: En el reto 2, se redujo el tiempo en un 20%, al no existir iteración

Si bien se observa que los tiempos de ejecución disminuyen en promedio respecto al reto 1, estamos hablando de diferencias de unidades de milisegundos, y como hay cierto margen de error en la medición, la diferencia es casi despreciable.